Operating System Final

Spring 2025 Final

1#why do we need virtual memory? what are the goals of memory virtualization?

Virtual memory is needed because:

1. Physical RAM is limited

Programs are often larger than the available RAM.
Virtual memory lets the OS run programs **even if they don't all fit** in RAM at once.

2. Multiple programs need to run at the same time

If every program used physical addresses directly, they would collide and overwrite each other.
Virtual memory gives **each program its own private memory space**, preventing conflicts.

3. It improves system stability and security

Virtual memory ensures:

- One program can't read or modify another program's data.
- Crashes in one program won't affect others.

4. It simplifies programming

Programmers don't need to worry about:

- exact physical memory locations
- memory fragmentation
- loading/unloading parts of the program manually

The OS handles all of it.

5. It allows the use of secondary storage (disk) as extra memory

Pages that are not actively used can be moved to disk ("paging").
This increases the **effective available memory** beyond physical RAM.

Goals of Memory Virtualization

Memory virtualization aims to achieve these key goals:

1. Transparency

Programs should behave as if:

- they have full access to a large, continuous memory,
- starting from address 0,
- even though physically this isn't true.

The program should not know or care that virtual memory exists.

2. Protection & Isolation

The system ensures:

- One process can't access another process's memory.
- Memory access violations are caught by the OS.

This improves **security and stability**.

3. Sharing

Some memory regions (like shared libraries or OS code) can be safely shared between processes, saving RAM.

4. Efficiency

The OS tries to:

- minimize swapping,
- use physical RAM effectively (e.g., through paging),
- reduce access time.

Goal: **Fast performance with minimal overhead**.

5. Flexibility

Virtual memory allows:

- dynamic memory allocation,
- growing stacks and heaps,
- loading only parts of a program as needed (demand paging).

Summary Table

Feature	Why It's Needed	Goal It Achieves
Virtual address space	RAM is small	Transparency
Private memory per process	Avoid conflicts	Protection
Paging to disk	Run large apps	Efficiency
Shared memory regions	Save RAM	Sharing
Simplified programming	Hide physical memory details	Transparency & Flexibility

2#what are the ways of allocating memory by a user?

Memory can be allocated in **three main ways**:

1. Static Allocation (Compile-Time Allocation)

What it means

Memory size is fixed **before** the program runs — at compile time.

Where it is stored

In **static/global** region.

Examples

- Global variables
- Static variables
- Constant arrays

```
int x = 10;
static int count;
```

Characteristics

- Size cannot change during runtime.
- Fastest allocation.
- Exists throughout the program's execution.

2. Automatic Allocation (Stack Allocation)

What it means

Memory is allocated automatically when a function starts and freed when the function ends.

Where it is stored

On the **stack**.

Examples

- Local variables
- Function arguments

```
void fun() {
   int a = 5;      // stack allocation
}
```

Characteristics

- Very fast
- Lifetime = function execution
- Size must be known at compile-time (mostly)
- Limited space (stack overflow possible)

3. Dynamic Allocation (Runtime Allocation)

What it means

Memory is allocated **at runtime** using OS or language library functions.

Where it is stored

On the **heap**.

Examples in C

```
int *p = malloc(sizeof(int));
free(p);
```

Functions used:

- malloc()
- calloc()
- realloc()

- free()

Characteristics

- Size can grow or shrink as needed
- Flexible but slower
- Must be manually freed (in C)
- More prone to memory leaks if misused

Summary Table

Allocation Type	When Allocated	Where Stored	Who Manages It	Flexibility
Static	Compile time	Global/Static region	Compiler	Fixed size
Automatic (Stack)	Function entry	Stack	Compiler/Runtime	Limited
Dynamic (Heap)	Runtime	Heap	Programmer/Allocator	Very flexible

Short Answer for Exams

Memory can be allocated in three ways:

1. **Static Allocation** – done at compile time for global and static variables.
2. **Automatic (Stack) Allocation** – done when functions execute for local variables.
3. **Dynamic Allocation (Heap)** – done during runtime using functions like malloc(), calloc(), and freed using free().

3#Describe the two simple aspects of file system and also illustrate a simple file system with proper figure.

✅ Two Simple Aspects of a File System

A file system has **two basic aspects**:

1. Data Management Aspect

This part deals with **how data is actually stored** on the storage device.

It includes:

- How files are broken into blocks
- How blocks are placed on disk
- How free space is tracked
- How data is read/written

Responsibilities:

- Storing data efficiently
- Ensuring no two files overwrite each other
- Maintaining free–block lists or bitmaps
- Managing fragmentation

Think: physical organization of data.

2. Directory Management Aspect

This part deals with **how files are organized so users can find them**.

It includes:

- Directory structure (tree, single-level, multi-level)
- File names and extensions
- File attributes (size, type, protection)
- Mapping file names to disk blocks

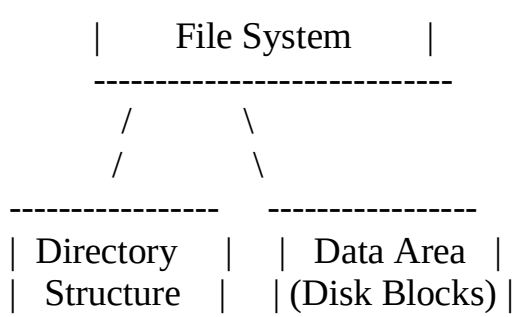
Responsibilities:

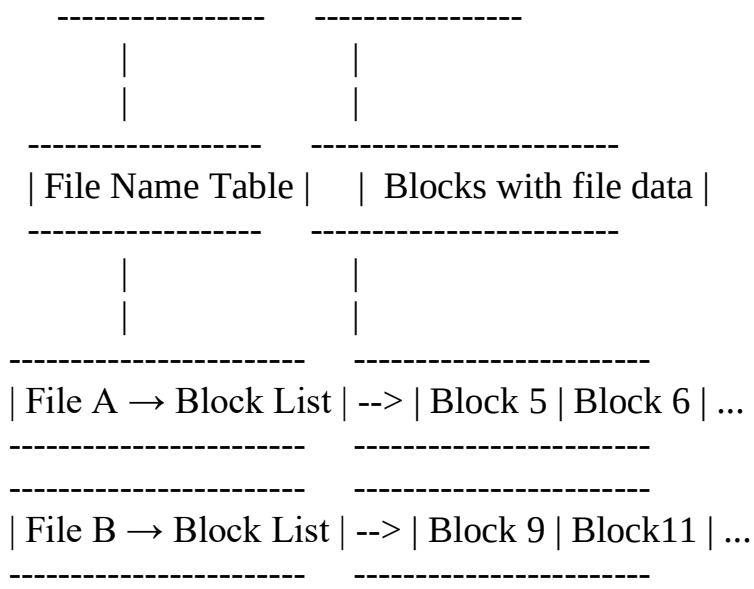
- Providing an easy way for users/programs to locate files
- Maintaining file metadata
- Access control (permissions)

Think: logical organization of files.

Simple File System – Illustrated

Below is a clean **ASCII diagram** showing how a simple file system works:





Explanation of the Diagram

Directory Structure

- Contains **file names** and **pointers** to the data blocks.
- Example:
  - File A → {5, 6}
  - File B → {9, 11}

Data Area

- Contains actual data stored in disk blocks.

Mapping

The file system connects **logical names** (File A, File B) to **physical storage** (Block 5, Block 6, etc.).

Short Answer for Exams

A file system has two main aspects:

1. **Data Management Aspect** – Handles physical storage of file data, block allocation, free space management, and read/write operations.
2. **Directory Management Aspect** – Manages file names, directories, attributes, and maps file names to their corresponding data blocks.

A simple file system consists of a directory structure that stores file names and metadata, and a data area that holds actual file contents organized in blocks.

4#Explain a simple device model with figure.

Simple Device Model (Explanation)

A **device model** explains how the operating system views and manages hardware devices (like keyboards, disks, printers, etc.).
Even though devices are different internally, the OS uses a **common structure** to control them.

A simple device model contains **three main components**:

1. Device Controller

- A small hardware component inside the computer.
- Controls the physical device.
- Converts high-level OS commands into low-level hardware actions.
- Manages data transfer between device and memory/CPU.

2. Device Driver

- A software layer inside the OS.
- Acts as a translator between the **OS** and **device controller**.
- Provides a standard interface so OS can work with different hardware.

3. The Device (Hardware)

- The actual physical component, such as:
  - Keyboard
  - Mouse
  - Hard disk
  - Printer
  - Network card

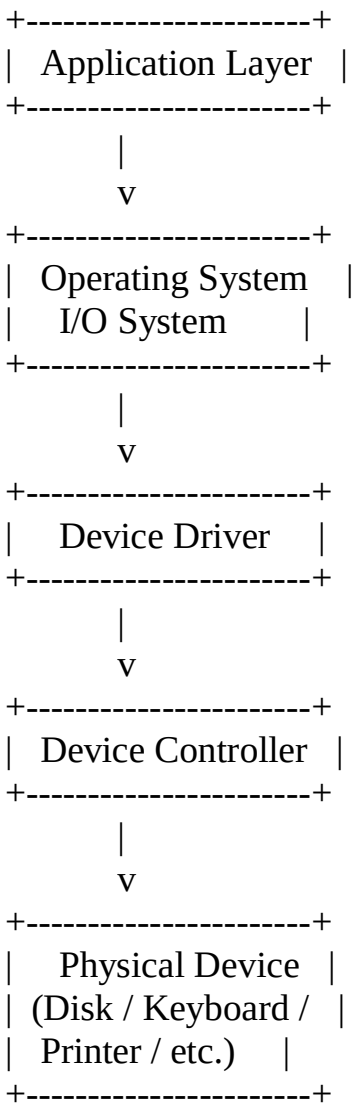
It executes operations like input, output, reading, writing, etc.

Flow in the Device Model

1. **Application Program** requests I/O.
2. Request goes to **OS I/O subsystem**.
3. The **device driver** sends commands to the **device controller**.
4. The **device controller** operates the actual hardware.

5. Data is sent back through the same path.

Simple Device Model – Figure



Short Exam Answer

A simple device model consists of the **device driver**, **device controller**, and the **physical device**.
The driver translates OS commands into device-specific instructions, the controller executes them at hardware level, and the device performs the actual I/O operations.
This model allows the OS to interact with different devices using a uniform interface.

If you want, I can also explain:

- I/O interrupt model

- DMA model
- Memory-mapped I/O vs port-mapped I/O
- Buffered vs unbuffered I/O

Just tell me!

You said:

5#Describe the structure of the inode of a file.

Structure of the Inode of a File

An **inode** (index node) is a data structure used in **UNIX/Linux file systems** (like ext2, ext3, ext4) to store **metadata** about a file.

➞ Important point:
An inode does NOT store the file name.
Names are stored in directories, which refer to inode numbers.

Fields Inside an Inode

A typical inode contains the following information:

1. File Type

Specifies what the inode represents:

- regular file
- directory
- symbolic link
- special device file

2. File Permissions

Defines who can read, write, and execute the file:

- owner permissions
- group permissions
- others' permissions

(Shown as rwxr-xr--.)

3. Owner Information

- **UID** → User ID of file owner

- **GID** → Group ID of file group

4. File Size

- Total size of the file (in bytes)

5. Timestamps

Usually three:

- **atime** → Last access time
- **mtime** → Last modification time
- **ctime** → Last metadata change time

(Some systems also add a creation time.)

6. Link Count

Number of directory entries pointing to the same inode.
(Used for hard links.)

7. Data Block Pointers

The **most important part** of the inode.

These pointers show where the actual file data is stored on disk.

They include:

- **12 direct pointers** → point directly to data blocks
- **1 single indirect pointer** → points to a block that contains more pointers
- **1 double indirect pointer** → points to a block that contains pointers to pointer blocks
- **1 triple indirect pointer** → used for very large files

This structure efficiently supports small, medium, and very large files.

8. File Flags

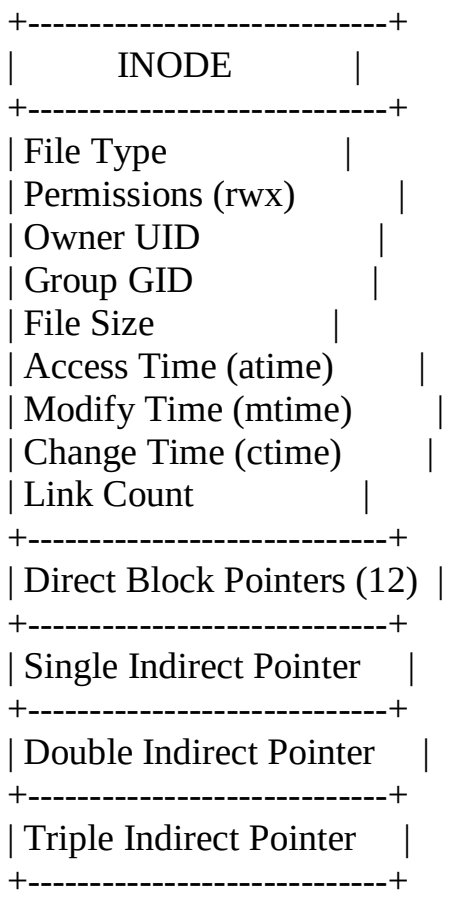
Used for special controls (e.g., append-only, immutable).

9. Reference Count / Generation Number

Used internally by filesystem operations.

Simple Figure of Inode Structure

You can draw this in your answer script:



Short Exam Answer

An inode is a filesystem data structure that stores metadata about a file, such as file type, permissions, owner IDs, size, timestamps, link count, and pointers to the file's data blocks. It contains 12 direct pointers and several indirect pointers (single, double, and triple), which together map a file to its physical disk blocks. File names are not stored in the inode but in directory entries that reference inode numbers.

6#Explain the two mechanisms of how OS read or write to registers.

Two Mechanisms of Reading/Writing Device Registers

Operating systems communicate with hardware devices through **device registers**, which control the device and exchange small pieces of data.

There are **two main mechanisms**:

1. Memory-Mapped I/O (MMIO)

Concept

- The device registers are **mapped into the normal physical memory address space**.
- The CPU reads/writes them **just like reading/writing RAM**.
- Standard load/store instructions (MOV, LD, ST) are used.

How It Works

- A portion of the address space is reserved for I/O.
- When the CPU writes to that address, it actually writes to the device register.
- No special I/O instructions are required.

Advantages

- Simple, uses regular memory instructions
- Can use full CPU addressing modes
- Fast for modern CPUs

Disadvantages

- I/O addresses consume parts of the memory address space
- Caches must be carefully managed (I/O regions are non-cacheable)

2. Port-Mapped I/O (PMIO) (also called Isolated I/O)

Concept

- Instead of sharing memory space, device registers live in a **separate I/O address space**.
- Special CPU instructions are used to access them.

Instructions Used

Common for x86:

- IN port → read from a port
- OUT port → write to a port

How It Works

- The OS issues IN/OUT instructions with a port number.
- The CPU routes these instructions to the I/O bus instead of memory.

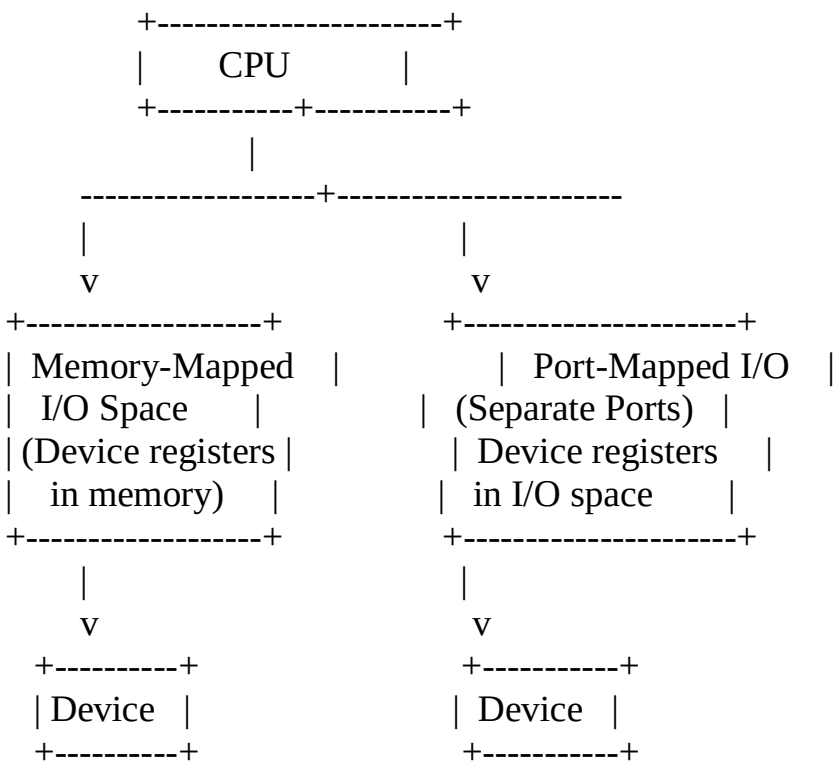
Advantages

- Does not consume memory address space
- Clear separation between memory and I/O

Disadvantages

- Only limited addressing modes
- Only available on CPUs that support special I/O instructions
- Less flexible than MMIO

Simple Figure: Two Mechanisms



Short Exam Answer

The OS can access device registers using **two mechanisms**:

1. **Memory-Mapped I/O (MMIO):** Device registers are mapped into the physical memory address space. The CPU uses regular load/store instructions to read/write them.
2. **Port-Mapped I/O (PMIO):** Device registers reside in a separate I/O address space. The CPU uses special I/O instructions (e.g., IN, OUT) to access them.

These two mechanisms define how the OS sends commands and receives data from hardware devices.